

Spark Performance Optimization Analysis

DSCC402 Final Project : Tweet Sentiment Analysis Pipeline

Name : Anshika Kothari

Pipeline Run Overview

The Tweet Sentiment Analysis pipeline ran successfully on May 8, 2026, completing in approximately 2 minutes and 4 seconds across four layers. Looking at the performance metrics from the Databricks Pipeline run, the Bronze layer took the longest at 5.285 seconds, which makes sense since it is responsible for scanning S3 and ingesting raw tweet files using CloudFiles Auto Loader. The Silver layer followed at 2.243 seconds, Gold at 1.545 seconds, and the Application layer materialized view refresh at 2.725 seconds. While these numbers look fast on an incremental run (where no new data was added), the pipeline initially processed around 50,000 tweets, and at that scale, certain performance bottlenecks become relevant. Below are three areas I identified and what can be done to improve them.

1. Shuffle : Mention Explosion in the Silver Layer

The Silver layer does something interesting: it takes each tweet and explodes it into multiple rows - one row per @mention found in the tweet text. So a tweet with three mentions becomes three rows. This is called a fan-out operation, and it significantly increases the row count going into the downstream layers. The problem is that when the Application layer later groups all these rows by mention to count positive and negative sentiments, Spark has to shuffle all the data across partitions to bring matching mentions together. This shuffle is expensive - it moves data over the network between executors and is one of the most common performance bottlenecks in Spark pipelines.

A simple fix would be to repartition the data by the mention column right after the explode step. This way, all rows with the same mention already land on the same partition, and the downstream groupBy becomes much cheaper since there is little to no data movement needed. This kind of change can reduce shuffle overhead by around 20 to 40 percent, especially as the dataset grows with daily tweet ingestion.

2. Serialization : ML Model UDF in the Gold Layer

The Gold layer loads a DistilBERT sentiment model from Unity Catalog and wraps it in a Spark UDF to run predictions on every tweet. This is where things get tricky performance-wise. Standard Python UDFs in Spark work row by row. For every single row, Spark has to serialize the data from its internal Java format, send it to a Python process, run the model, and then serialize the result back. When you multiply this overhead across tens of thousands of tweets, it

adds up quickly. The model itself is not the slow part - the constant back-and-forth serialization is.

The better approach is to use a Pandas UDF instead. Pandas UDFs work on entire batches of data at once using Apache Arrow, which is a highly efficient columnar memory format. This means instead of serializing row by row, Spark sends a whole chunk of data to Python in one shot, the model runs inference on the whole batch, and the results come back together. In practice, this can make ML inference UDFs anywhere from 5 to 10 times faster, which is a significant gain when running daily on a growing dataset.

3. Storage : Small Files in Delta Tables

Every time the pipeline runs, the streaming writes create new Parquet files in the Delta tables. On the first full run with 50,000 tweets, Spark likely created many small files across the Bronze, Silver, and Gold tables. Over time, with daily scheduled runs adding more data, this problem compounds - the tables end up with hundreds of tiny files instead of a few well-sized ones. This matters because when Spark reads a Delta table, it has to open and scan each file individually. Many small files means a lot of overhead just in file listing and metadata operations, before any actual data processing even begins.

The fix is to periodically run the OPTIMIZE command on the Delta tables, which compacts all those small files into larger, more efficient ones. Combining this with ZORDER on frequently filtered columns like mention or predicted_sentiment means Spark can skip large portions of data entirely when running aggregation queries. Enabling auto - optimize on the tables so this happens automatically during writes is an even better long-term solution. These storage optimizations can improve query scan performance by 30 to 60 percent on tables that have gone through multiple write cycles.

Summary

Area	Problem	Fix	Expected Gain
Shuffle	Expensive groupBy after mention explode	Repartition by mention post-explode	20–40% shuffle reduction
Serialization	Row-by-row Python UDF for ML inference	Switch to Pandas UDF with Arrow batching	5–10x inference speedup
Storage	Small file accumulation across daily runs	OPTIMIZE + ZORDER on Delta tables	30–60% scan improvement

All three of these are practical, low-effort changes that would meaningfully improve pipeline performance at scale without requiring any architectural redesign.

Jobs & Pipelines > Final_Project_Job > Run 82593864301959 >

Final_Project_pipeline run Succeeded

Edit task Repair task

Final project

Open pipeline details

Details

Job ID 292746723309046

Job run ID 82593864301959

Task run ID 221227119128122

Run as akothar9@ur.rochester.edu

Status Completed

Run type Refresh all

Tables 4 Performance 4

Filter by table

View all in query history

Statement	Started At	Duration	Rows read	Bytes read	Bytes written
REFRESH MATERIALIZED VIEW workspace.default.gold_tweet_aggregations	May 08, 2026, 10:03 AM	2 s 725 ms	0	0 B	0 B
REFRESH STREAMING TABLE workspace.default.tweets_gold />	May 08, 2026, 10:02 AM	1 s 545 ms	0	0 B	0 B
REFRESH STREAMING TABLE workspace.default.tweets_silver />	May 08, 2026, 10:02 AM	2 s 243 ms	0	0 B	0 B
REFRESH STREAMING TABLE workspace.default.tweets_bronze />	May 08, 2026, 10:02 AM	5 s 285 ms	0	0 B	0 B

May 08, 2026, 10:01 AM 2m 4s Refresh all

Tables 4 Performance 4

Filter by table

View all in query history

Statement	Started At	Duration	Rows read	Bytes read	Bytes written
REFRESH MATERIALIZED VIEW workspace.default.gold_tweet_aggregations	May 08, 2026, 10:03 AM	2 s 725 ms	0	0 B	0 B
REFRESH STREAMING TABLE workspace.default.tweets_gold />	May 08, 2026, 10:02 AM	1 s 545 ms	0	0 B	0 B
REFRESH STREAMING TABLE workspace.default.tweets_silver />	May 08, 2026, 10:02 AM	2 s 243 ms	0	0 B	0 B
REFRESH STREAMING TABLE workspace.default.tweets_bronze />	May 08, 2026, 10:02 AM	5 s 285 ms	0	0 B	0 B

Query History

Refresh

User Last 30 days Compute Duration Status Statement Source Statement ID

Pipeline Update ID 46b37cf4-f504-4bf2-9a9d-adbaf8808ba0 4 queries Reset filters

Query	Started at	Duration	Source	Compute	User
REFRESH MATERIALIZED VIEW workspace.default.gold_tweet_aggregations	5/08/2026, 10:...	2.73 s	Final_P...ect_Job / ... / Final project	Pipeline comp...	akothar9@ur.roche
REFRESH STREAMING TABLE workspace.default.tweets_gold />	5/08/2026, 10:...	1.54 s	Final_P...ect_Job / ... / Final project	Pipeline comp...	akothar9@ur.roche
REFRESH STREAMING TABLE workspace.default.tweets_silver />	5/08/2026, 10:...	2.24 s	Final_P...ect_Job / ... / Final project	Pipeline comp...	akothar9@ur.roche
REFRESH STREAMING TABLE workspace.default.tweets_bronze />	5/08/2026, 10:...	5.29 s	Final_P...ect_Job / ... / Final project	Pipeline comp...	akothar9@ur.roche

Finished

akothar9@ur.rochester.edu · Pipeline compute

Copy URL Download

Query text

1 REFRESH MATERIALIZED VIEW workspace.default.gold_tweet_aggregation

Query profile is not available.